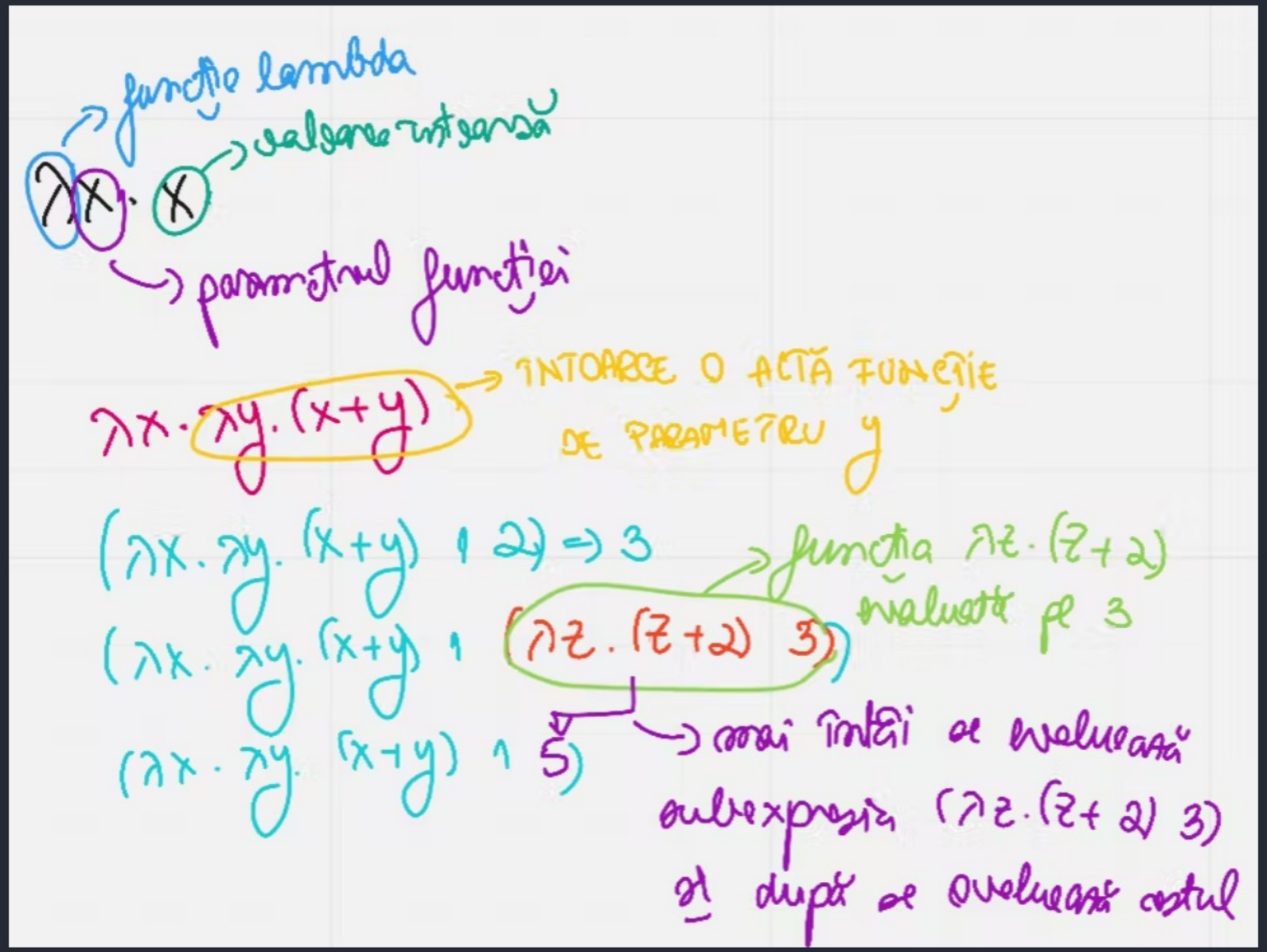
Mentimeter

Racket: Întârzierea evaluării



The screenshot couldn't be generated


funcție lambda
valoare returnată
parametrul funcției
λx. λy. (x+y)
ÎNTOARCE O ALTĂ FUNCȚIE
DE PARAMETRU y
(λx. λy. (x+y) 1 2) ⇒ 3
(λx. λy. (x+y) 1 (λz. (z+2) 3))
(λx. λy. (x+y) 1 5)
funcția λz. (z+2)
evaluată pe 3
mai întâi se evaluează
subexpresia (λz. (z+2) 3)
și după se evaluează restul

Exemplu

Mentimeter

Evaluare leneșă

- nu evaluăm parametrii până ce nu avem nevoie de valoarea lor
- parametrii sunt transmiși prin nume

Mentimeter

$(\lambda x.\lambda y.(x+y)\ 1\ (\lambda z.(z+2)\ 3))$

$(1 + (\lambda z.(z+2)\ 3))$

Încă nu trebuie calculată (NU EVALUEZ FUNCȚIA)

Trebuie să calculăm parametrii pt. a putea aplica + pe ei

$(1+5)$

6

Exemplu

Mentimeter

Evaluarea în Racket

- în mod normal este **aplicativă**
- putem obține evaluări lenești prin închideri funcționale sau promisiuni

Mentimeter

Evaluare leneșă cu închideri

- întoarcem o funcție care încă nu este evaluată
- ex: (define pow (lambda (x) (lambda () (* x x))))
- (pow 2) = închidere funcțională
- ((pow 2)) evaluează expresia la 4

Mentimeter

Evaluare leneșă cu promisiuni

- **delay** întârzie evaluarea, **force** forțează evaluarea
- ex: (define pow (lambda(x) (* x x)))
-
- (pow 2) = promisiune
- (force (pow 2)) = 4

Mentimeter

Super important

- force evaluează o singură dată expresia
- la primul apel se evaluează și se salvează valoarea (un fel de cache)
- dacă încercăm să forțăm o nouă evaluare a aceleiași promisiuni atunci se întoarce valoarea salvată

Mentimeter

```
(define (print_pow x) (display x) (* x x))
(define lazy (delay (print_pow 1)))
(define copy lazy)
(+ (force lazy) (force copy))
```

Afișează 12

Mentimeter

Explicație

- (delay (print_pow 1)) întoarce o promisiune
- copy e o "referință" către aceeași promisiune
- la (force lazy) se evaluează promisiunea: se afisează 1 și se întoarce 1
- la (force copy) se întoarce direct 1 pentru că aceeași promisiune a fost deja evaluată

Mentimeter

Stream

- element curent
- generator care întoarce următorul element din stream

Mentimeter

Șirul constant de 1

- (define ones (cons 1 (delay ones)))
- echivalent cu (define ones (stream-cons 1 ones))

Mentimeter

Funcții pe stream-uri 0

- stream-cons <=> cons
- stream-first <=> car
- stream-rest <=> cdr

Mentimeter

Funcții pe stream-uri 1

- empty-stream <=> null/'()
- stream-empty? <=> null?
- stream-map <=> map (**aici accepta numai un stream, la liste puteau fi mai multe**)

Mentimeter

Funcții pe stream-uri 2

- stream-append <=> append
- stream-filter <=> filter

Mentimeter

stream-take s n

- un stream poate conține o infinitate de elemente
- vrem o funcție prin care să obținem numai primele n

Mentimeter

```
(define (stream-take s n)
  (cond
    [(zero? n) '()]
    [(stream-empty? s) '()]
    [else (cons (stream-first s) (stream-take (stream-rest s) (- n 1)))]))
```

Implementare

Mentimeter

Șirul numerelor naturale

- (define (naturals-from n) (stream-cons n (naturals (+ n 1))))
- (define naturals (naturals-from 0))

Mentimeter

Șirul lui Fibonacci

- f(n) = f(n-1) + f(n-2)
- putem rescrie șirul folosind calcule pe stream uri

Mentimeter

Idee: f = f + (stream-rest f)

- f(0) f(1) f(2) f(3) f(4) .... f(n-2) f(n-1) .... +
- f(1) f(2) f(3) f(4) f(5) .... f(n-1) f(n) .....
- ----------
- f(2) f(3) f(4) f(5) f(6) ... f(n) f(n+1) .....

Mentimeter

Probleme

- lipsesc primele 2 numere f(0), f(1) => le adăugăm în față cu stream-cons
- trebuie să calculăm suma a 2 stream-uri => ne facem funcție

Mentimeter

```
(define (add s1 s2)
  (stream-cons
   (+ (stream-first s1) (stream-first s2))
   (add (stream-rest s1) (stream-rest s2))))

(define fibo
  (stream-cons 0 (stream-cons 1 (add fibo (stream-rest fibo)))))
(stream-take fibo 10)
```

Implementare

Mentimeter

Șirul lui Eratostene

- începem de la 2 și alegem mereu primul element din șir drept divizor
- pentru fiecare număr ales (divizor) eliminăm toate numerele divizibile cu el și continuăm

Mentimeter

```
(define (sieve s)
  (let ([first (stream-first s)])
    (stream-cons
     first
     (sieve (stream-filter (lambda (x) (not (zero? (modulo x first)))) (stream-rest s))))))

(define primes
  (sieve (stream-rest (stream-rest naturals))))
```

Implementare



Mentimeter

Extras

Mentimeter

1. Reduceți expresia lambda $E = (\lambda x.(x \ \ (\lambda y.z \ \ x)) \ \ \lambda x.x)$

Mentimeter

1. Folosind evaluare normală, aduceți la forma normală expresia $(\lambda x.(\lambda y.(x\ y)\ \lambda z.(y\ z))\ x)$.

Mentimeter

1. `(λx y z.corp a b c)` este forma prescurtată de a scrie `(((λx.λy.λz.corp a) b) c)`.

   Fie următoarele definiții în Calcul Lambda:

   `true = λx y.x`

   `false = λx y.y`

   și fie operatorul logic:

   `op = λx y.(x y true)`

   Descrieți pas cu pas (nu efectuați mai multe β-reduceri deodată) comportamentul lui `op` pe valori de tip `true` sau/și `false`, astfel încât să puteți conchide la final că `op` se comportă ca unul dintre operatorii logici cunoscuți (și precizați care este operatorul a cărui identitate a fost ascunsă prin folosirea numelui "`op`").

Select Answer

Mentimeter

